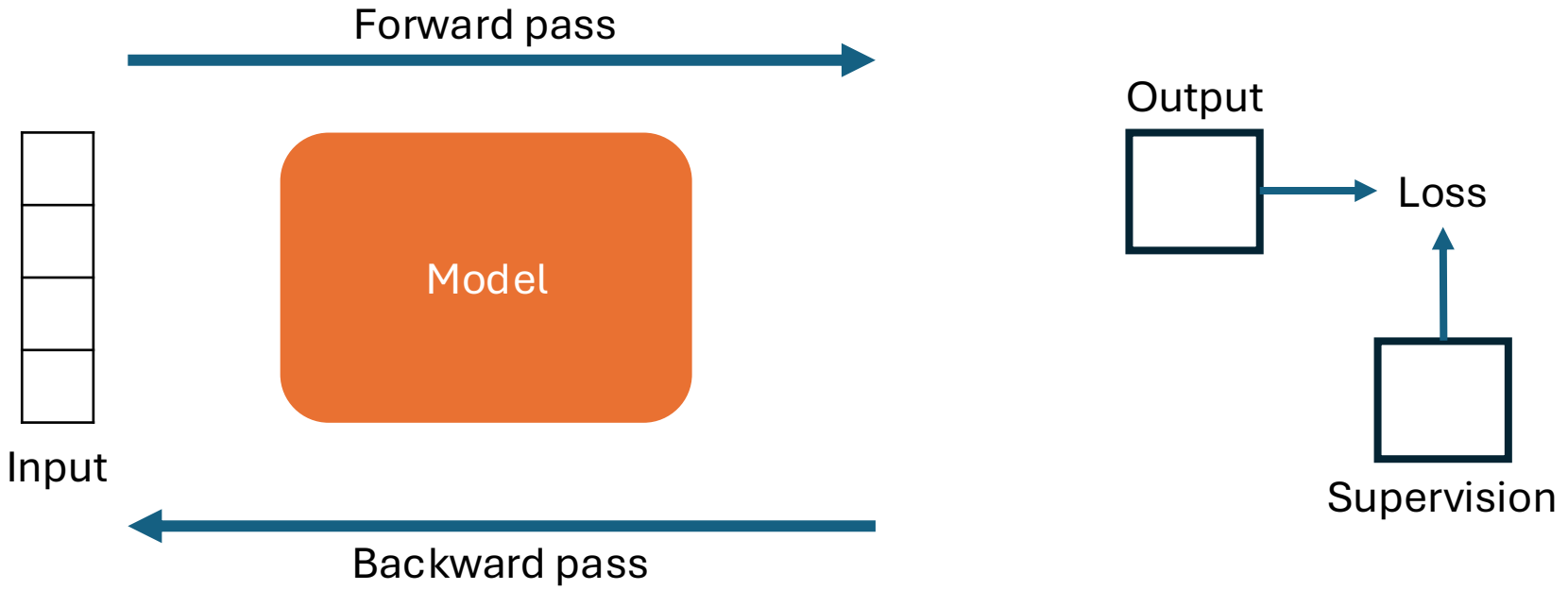
PyTorch Autograd

Amit Shmidov | Tutorial 3

This presentation largely rely on DL4CV@Weizmann 23’

Neural Network | High level pipeline



Neural Network | A python class

```
class MyNetwork(nn.Module):
    def __init__(self):                # runs once at construction
        super().__init__()
        # declare your layers
    def forward(self, x):              # runs every model(x) call
        # describe how data flows
        return ...
```

Contain learnable parameters

By subclassing nn.Module:

- `model.parameters()` sees every layer declared in `__init__`
- Recursive `.to(device)`, `.train()`, `.eval()` acros every sub-module

Building a Neural Network

```
INPUT_SIZE = 224
C = 10  # num classes

class MLP(nn.Module):

  def __init__(self):
    super(MLP, self).__init__()
    self.flatten = nn.Flatten()
    self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
    self.relu1 = nn.ReLU()
    self.fc2 = nn.Linear(256, 512)
    self.relu2 = nn.ReLU()
    self.fc3 = nn.Linear(512, C)

  def forward(self, x):
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu1(x)
    x = self.fc2(x)
    x = self.relu2(x)
    x = self.fc3(x)
    return x
```

Building a Neural Network

```
INPUT_SIZE = 224
C = 10  # num classes

class MLP(nn.Module):

  def __init__(self):
    super(MLP, self).__init__()
    self.flatten = nn.Flatten()
    self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
    self.relu1 = nn.ReLU()
    self.fc2 = nn.Linear(256, 512)
    self.relu2 = nn.ReLU()
    self.fc3 = nn.Linear(512, C)

  def forward(self, x):
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu1(x)
    x = self.fc2(x)
    x = self.relu2(x)
    x = self.fc3(x)
    return x
```

Initialization:

```
B = 10  # batch size

input = torch.randn(B, 1, INPUT_SIZE, INPUT_SIZE)
model = MLP()
```

Input shape: (B=10, 1, H=224, W=224)
Output shape: (B=10, C=10)

Building a Neural Network

```
INPUT_SIZE = 224
C = 10   # num classes

class MLP(nn.Module):

  def __init__(self):
    super(MLP, self).__init__()
    self.flatten = nn.Flatten()
    self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
    self.relu1 = nn.ReLU()
    self.fc2 = nn.Linear(256, 512)
    self.relu2 = nn.ReLU()
    self.fc3 = nn.Linear(512, C)

  def forward(self, x):
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu1(x)
    x = self.fc2(x)
    x = self.relu2(x)
    x = self.fc3(x)
    return x
```

Initialization:

```
B = 10   # batch size

input = torch.randn(B, 1, INPUT_SIZE, INPUT_SIZE)
model = MLP()
```

Input shape: (B=10, 1, H=224, W=224)
Output shape: (B=10, C=10)

Run with CPU

```
output = model(input)
```

7.96 ms

Run with GPU

```
gpu_model = model.to('cuda')
gpu_input = input.to('cuda')

gpu_output = gpu_model(gpu_input)
```

0.58 ms

Building a Neural Network

```
INPUT_SIZE = 224
C = 10  # num classes

class MLP(nn.Module):

  def __init__(self):
    super(MLP, self).__init__()
    self.flatten = nn.Flatten()
    self.fc1 = nn.Linear(INPUT_SIZE*INPUT_SIZE, 256)
    self.relu1 = nn.ReLU()
    self.fc2 = nn.Linear(256, 512)
    self.relu2 = nn.ReLU()
    self.fc3 = nn.Linear(512, C)

  def forward(self, x):
    x = self.flatten(x)
    x = self.fc1(x)
    x = self.relu1(x)
    x = self.fc2(x)
    x = self.relu2(x)
    x = self.fc3(x)
    return x
```

Using `nn.Sequential`:

```
seq_model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(INPUT_SIZE*INPUT_SIZE, 256),
    nn.ReLU(),
    nn.Linear(256, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
)

output = seq_model(input)
```

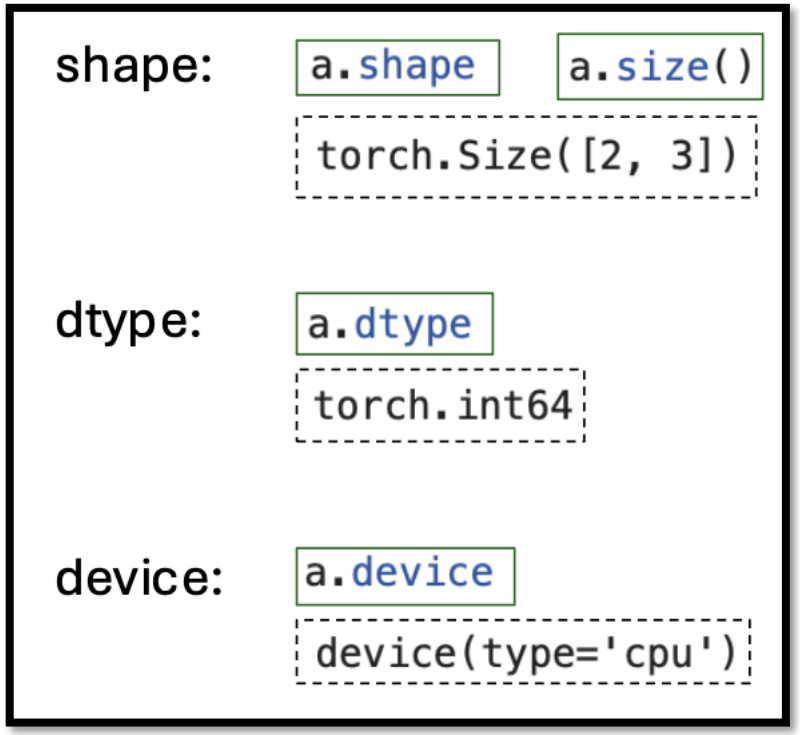
Network Training | Reminder

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD update: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

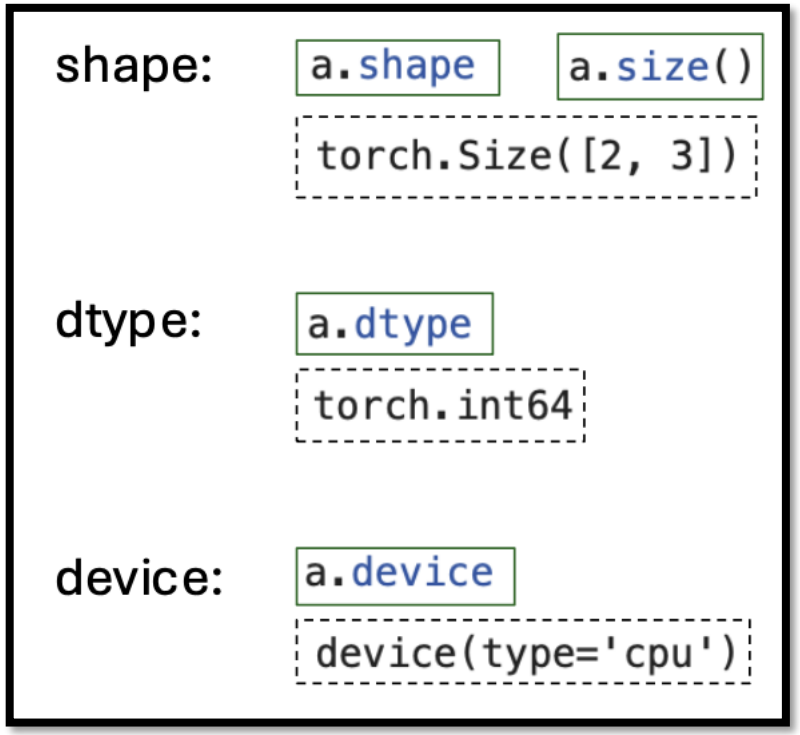
Network Training | Reminder

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD update: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

Tensor Attributes



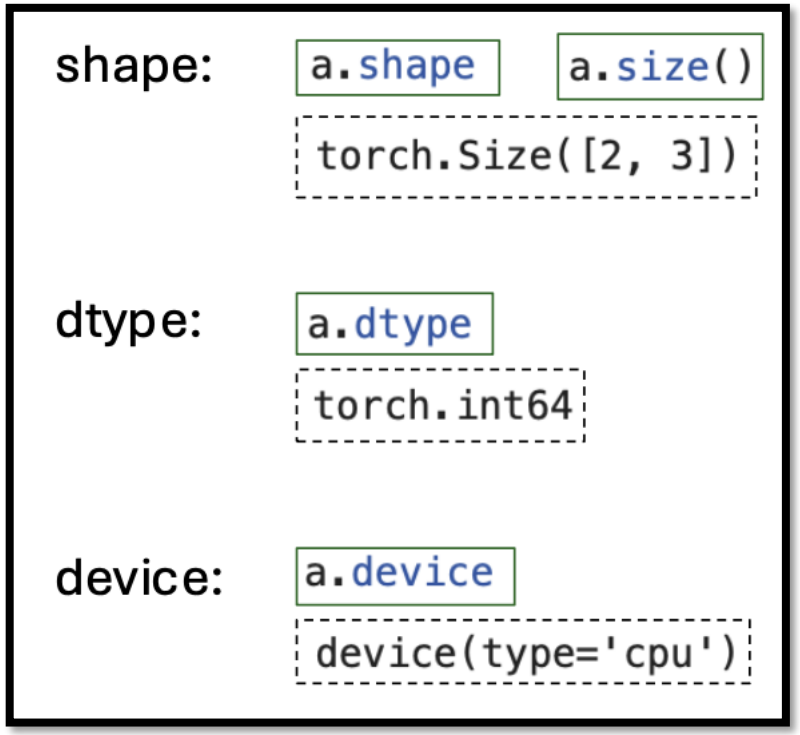
Tensor Attributes



`.grad` – gradient for this tensor.

`.grad_fn` – reference for a function used to obtain it.

Tensor Attributes



`.grad` – gradient for this tensor.

`.grad_fn` – reference for a function used to obtain it.

```
x = torch.tensor([2.])
w = torch.tensor([1.],
    requires_grad=True)
y = torch.tensor([3.])

y_hat = x * w
L = (y_hat - y) ** 2

L.backward()

print(w.grad)
print(x.grad)
```

-4

None

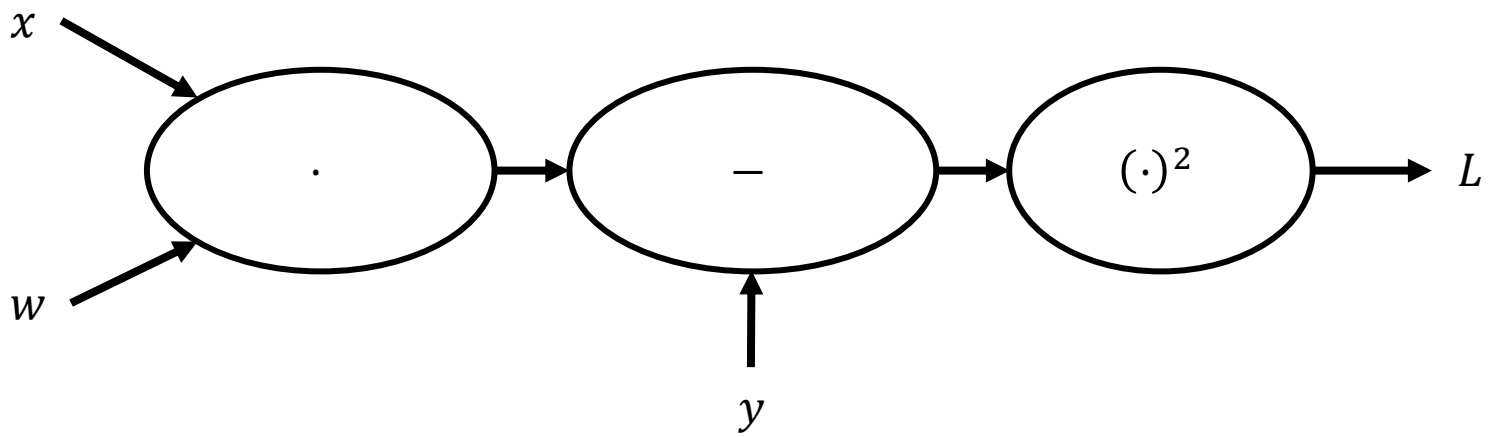
Computational Graph

DAG (Directed Acyclic Graph):

- Nodes correspond to operations or variables.
- Can be used to differentiate via the chain rule.

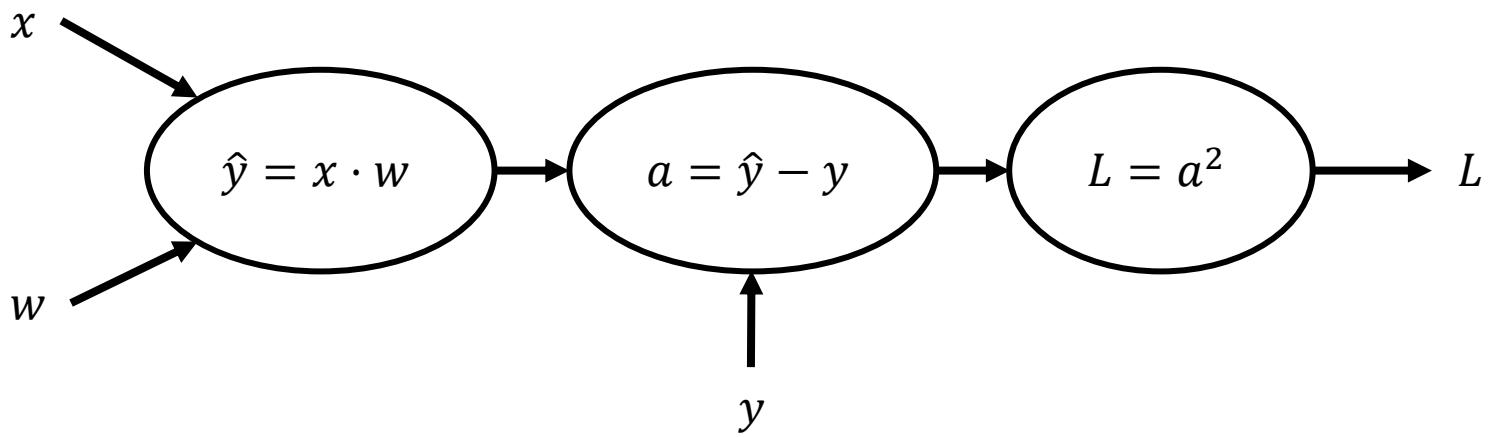
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



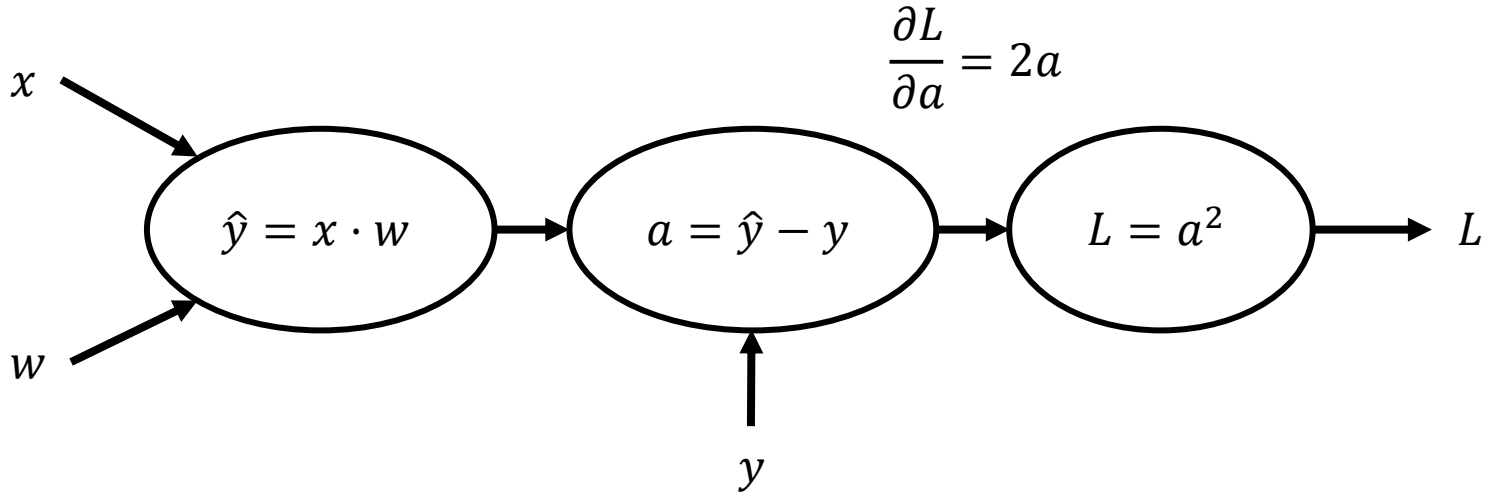
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



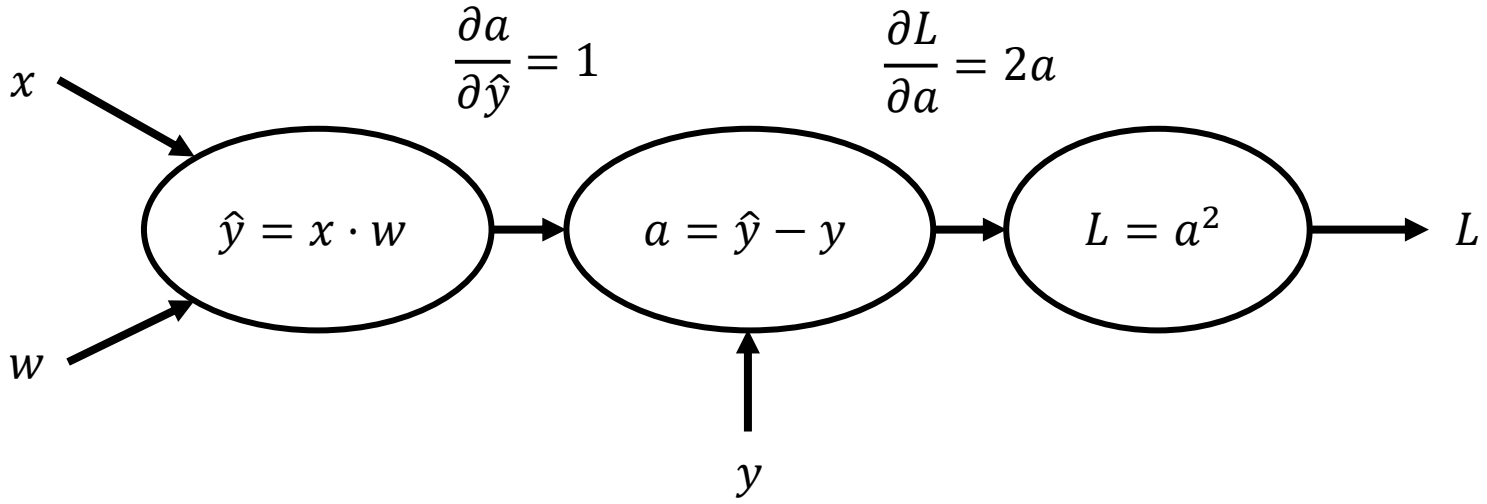
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



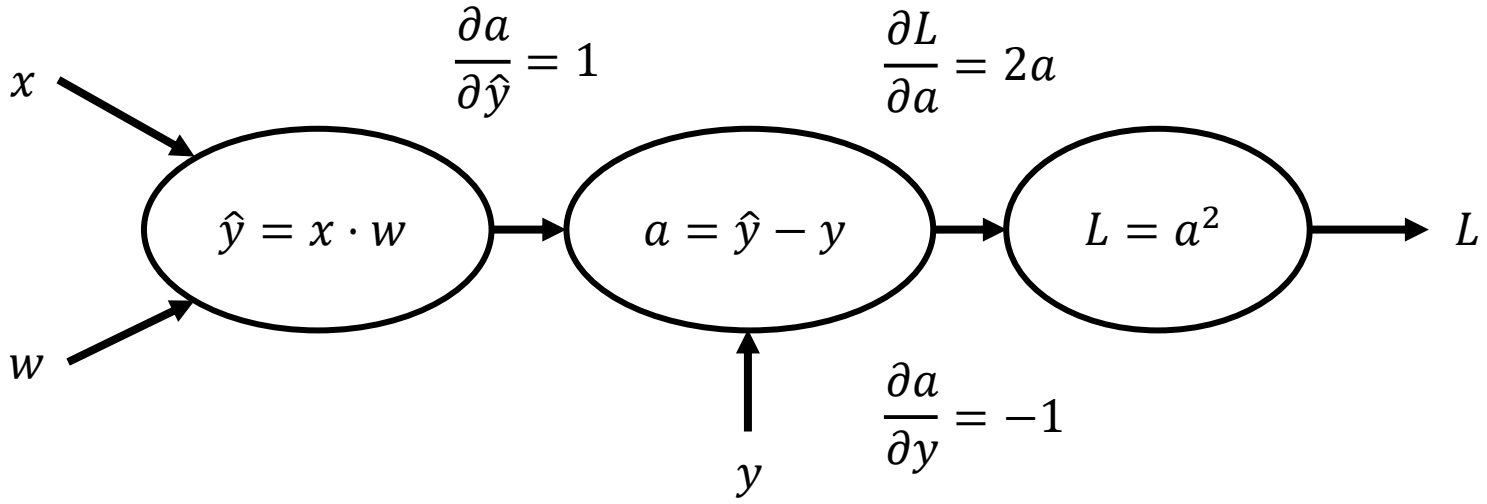
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



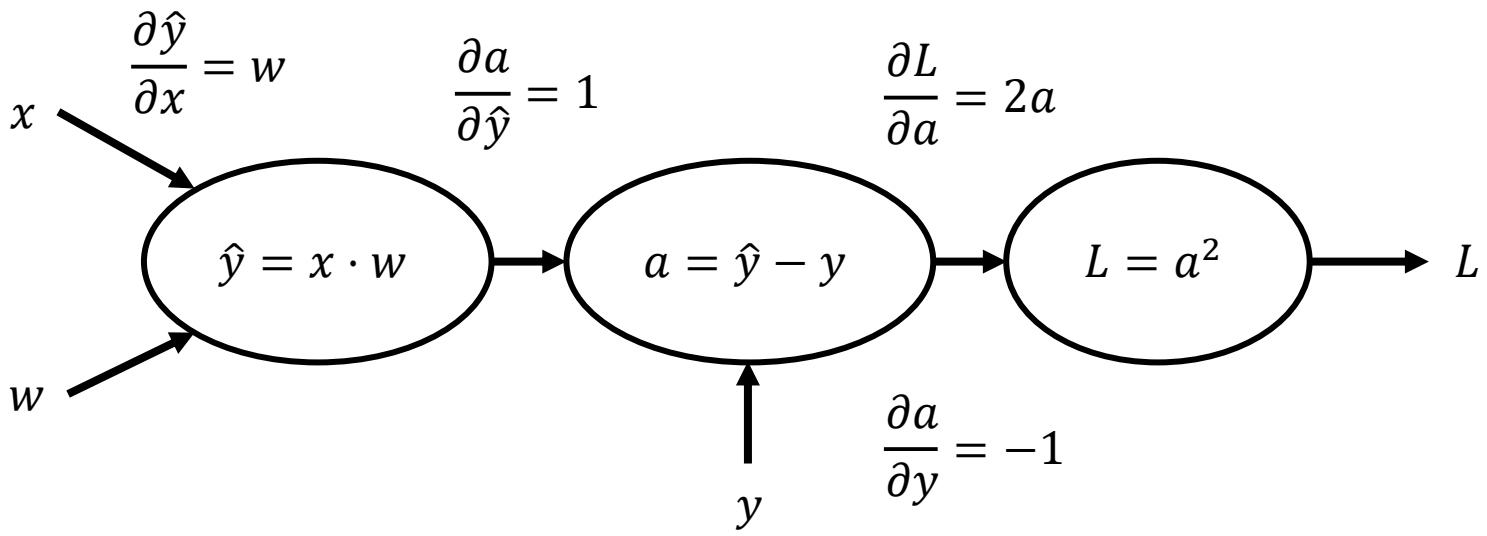
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



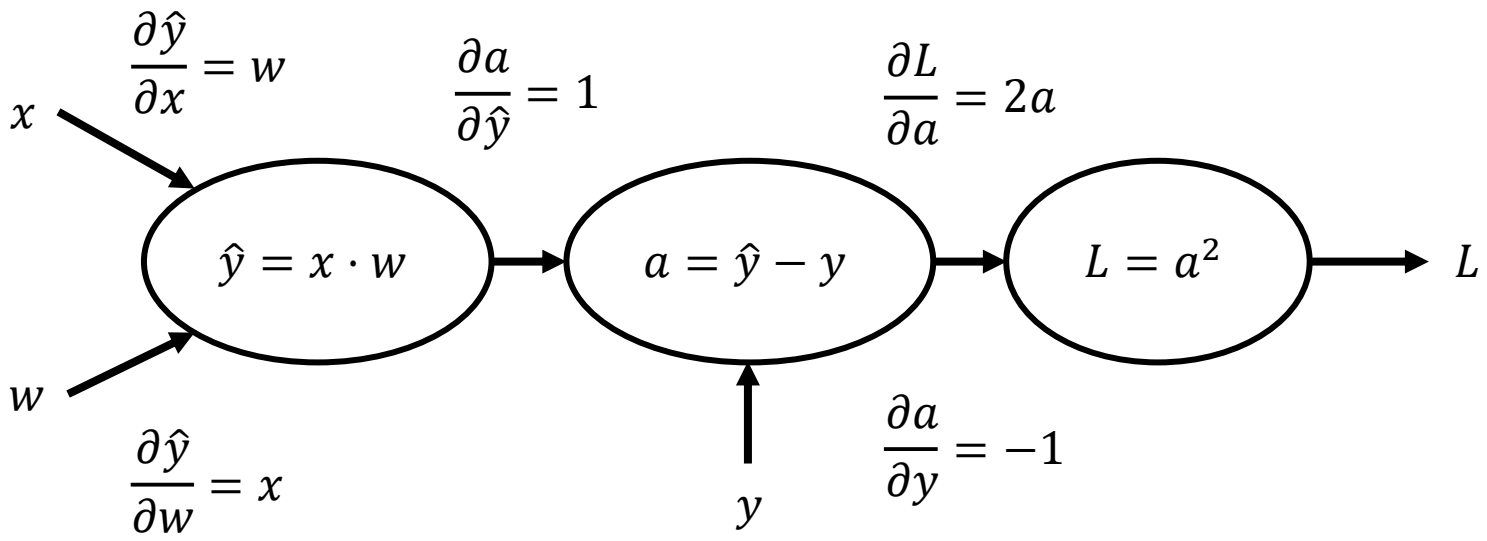
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



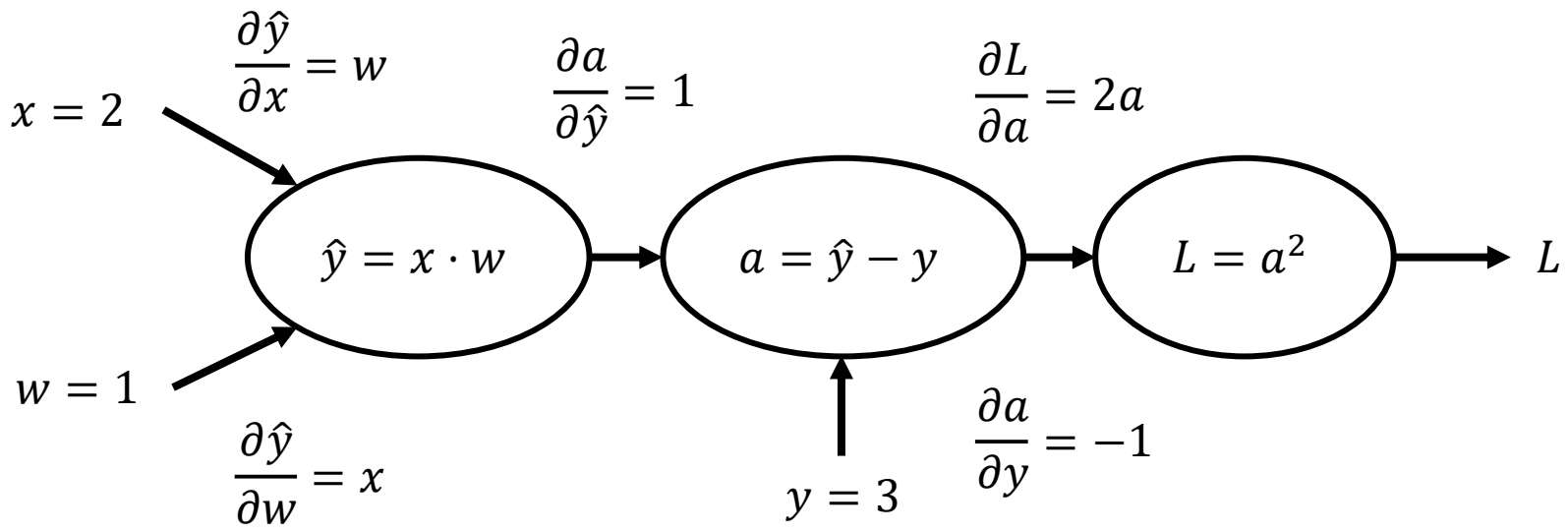
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



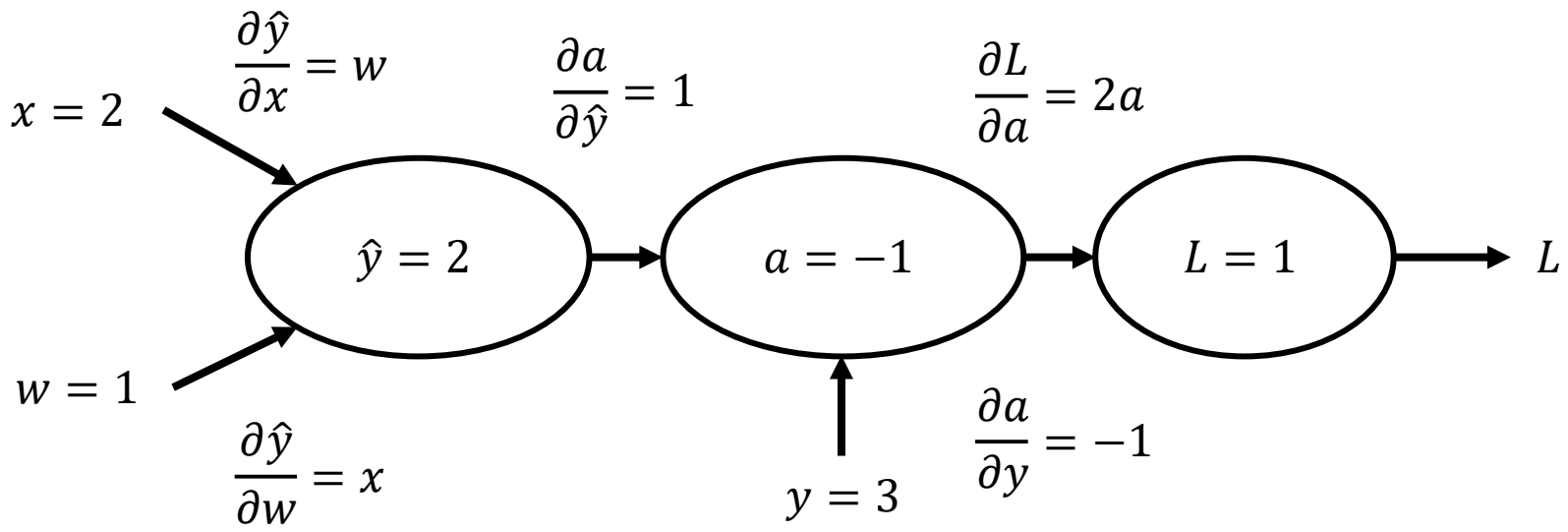
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



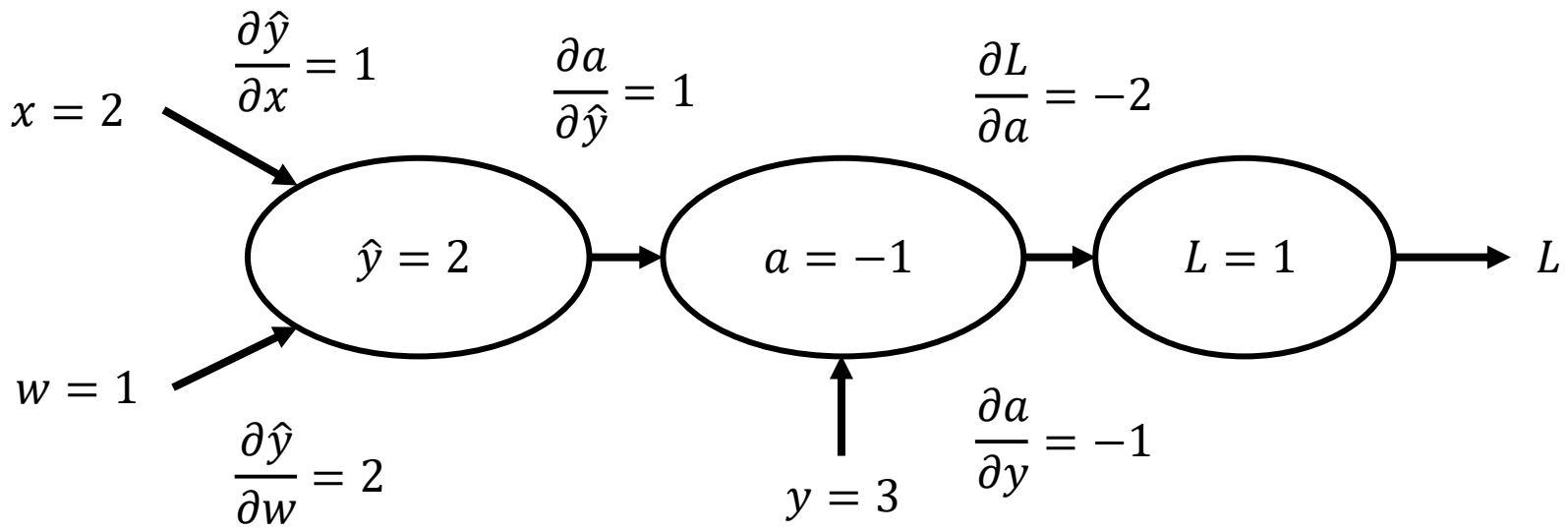
Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



Computational Graph

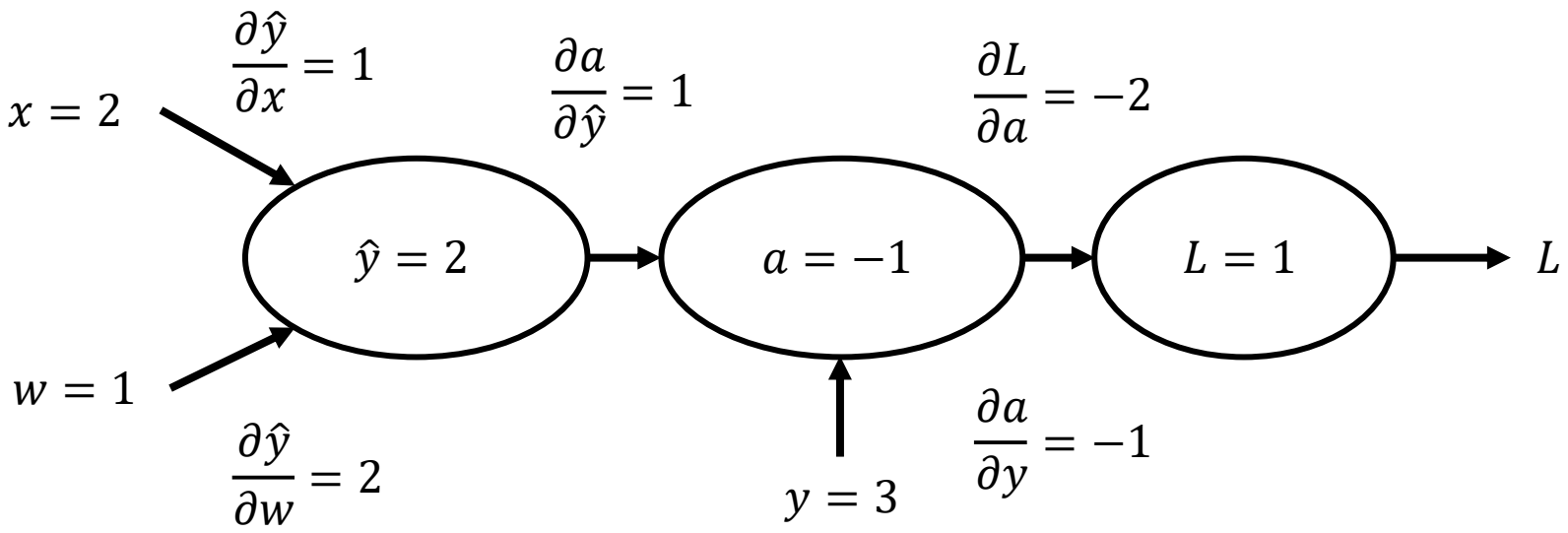
- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$



Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

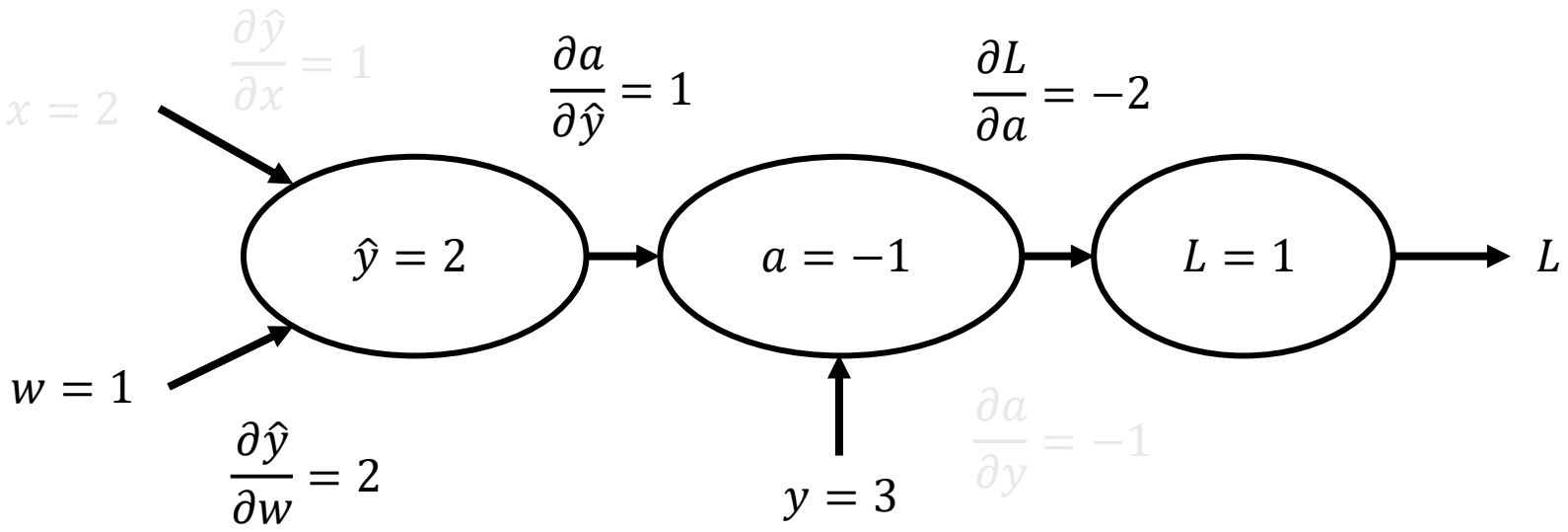
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w}$$



Computational Graph

- Data: $x, y \in R$
- Hypothesis: $\hat{y} = x \cdot w$
- Loss: $L = (\hat{y} - y)^2$
- SGD: $w \leftarrow w - \alpha \cdot \frac{\partial L}{\partial w}$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w} = -4$$



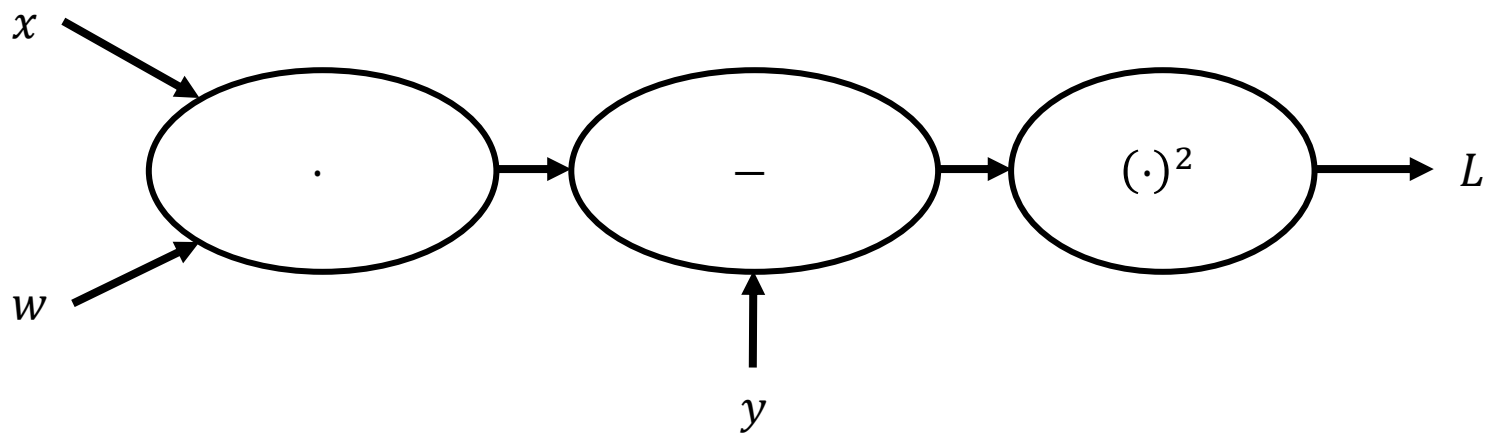
Computational Graph

Static graph – fixed structure, data is injected into a prebuilt graph.

Dynamic graph – structure is defined on the fly during execution.

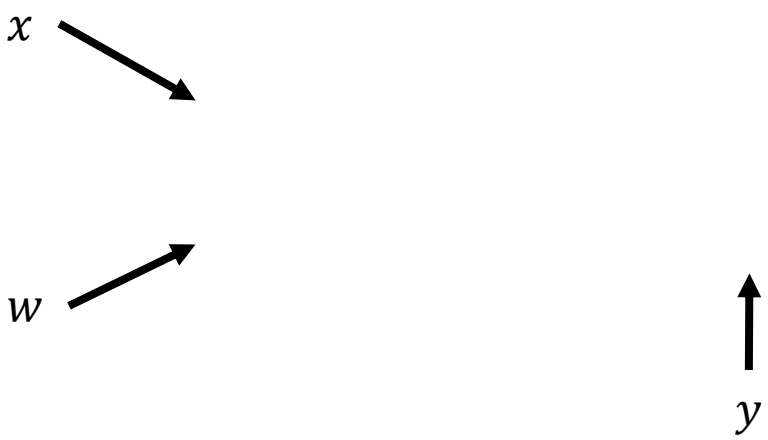
Static Graph

First define the graph, then inject data:



Dynamic Graph

Variables are defined, the graph is build during execution:

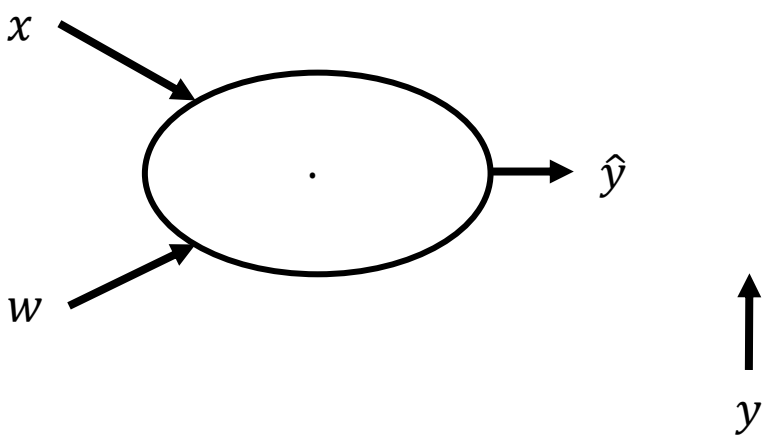


Code so far:

```
x, w, y
```

Dynamic Graph

Variables are defined, the graph is build during execution:

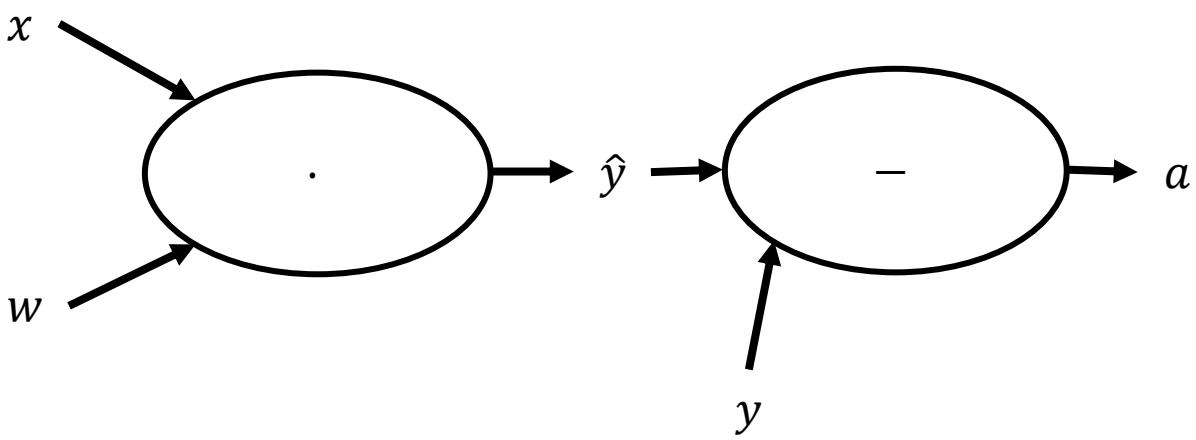


Code so far:

```
x, w, y
y_hat = x * w
```

Dynamic Graph

Variables are defined, the graph is build during execution:

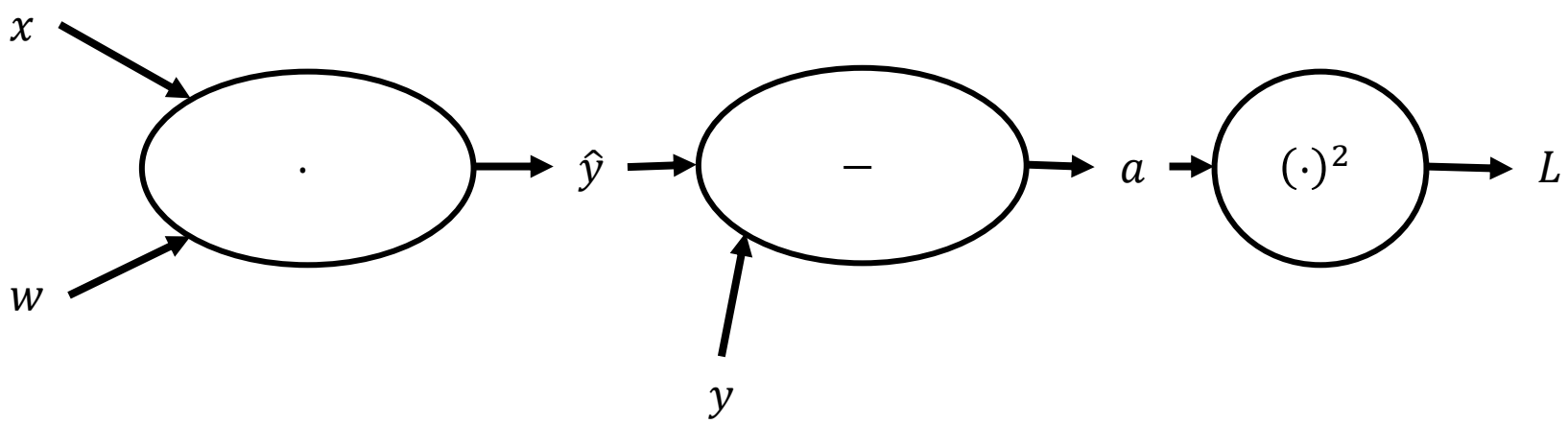


Code so far:

```
x, w, y
y_hat = x * w
a = y_hat-y
```

Dynamic Graph

Variables are defined, the graph is build during execution:

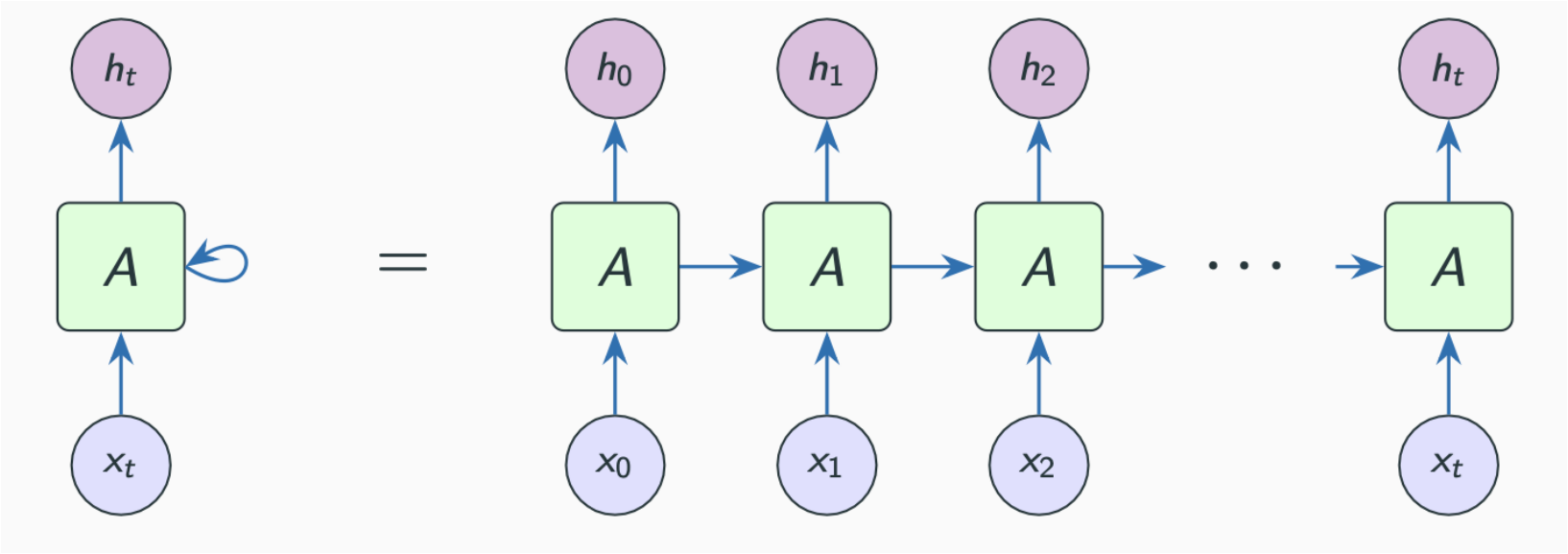


Code so far:

```
x, w, y
y_hat = x * w
a = y_hat-y
L = a ** 2
```

Dynamic Graph | Why?

Variable-lengh sequences:




```
def forward(self, words):                # len(words) varies per input
    h = torch.zeros(self.hidden_dim)
    for word in words:
        h = tanh(Wx @ word + Wh @ h)
    return self.out(h)
```

Dynamic Graph | Why?

Adaptive-depth networks:

```
def forward(self, x):
    h = self.stem(x)
    for layer in self.layers:
        h = layer(h)
        p = self.head(h).softmax(-1)
        if p.max() > 0.95:          # data-dependent branch
            return p
    return self.head(h)
```

Easy data points require less layers than hard ones.

Static Vs. Dynamic

Static	Dynamic
Define-and-run	Define-by-run
Slightly faster	flexible
TensorFlow < 2.0	PyTorch & TensorFlow ≥ 2

Computational Graph in PyTorch

- Dynamic
- Built during the forward pass
  - Unless `torch.no_grad()` was used
- Deleted during the backward pass
- Re-built on the next forward pass
- Gradients accumulate across `backward()` calls

Gradient Accumulation

Problem: you want `batch_size=128`, but only `batch_size=32` fits in GPU memory.

Solution: Run 4 forward + backward passes without `zero_grad()` and run `optimizer.step()` once.

```
ACCUM_STEPS = 4    # effective batch = 32 * 4 = 128

optimizer.zero_grad()
for i, (X, y) in enumerate(loader):
    loss = criterion(model(X), y) / ACCUM_STEPS   # rescale
    loss.backward()                               # accumulates
    if (i + 1) % ACCUM_STEPS == 0:
        optimizer.step()
        optimizer.zero_grad()
```

Disabling Gradients

When will we disable autograd?

On inference, validation or when computing statistics on inputs or weights. Anytime we do not need `backward()`.

```
with torch.no_grad():
    pred = model(x)
```

No graph is constructed. Tensors exist but can't be differentiated through

```
z = y.detach()
```

Returns a view that's inconnected to the graph. Useful when you want the value but cut the gradient path.

Disabling Gradients

When will we disable autograd?

On inference, validation or when computing statistics on inputs or weights. Anytime we do not need `backward()`.

```
with torch.no_grad():
    pred = model(x)
```

No graph is constructed. Tensors exist but can't be differentiated through

```
z = y.detach()
```

Returns a view that's inconnected to the graph. Useful when you want the value but cut the gradient path.

```
tensor.detach().cpu().numpy()
```

From Tensor back to NumPy

Optimizers

Create an SGD optimizer:

```
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)
```

Reset gradients

```
optimizer.zero_grad()
```

Take an optimization step

```
optimizer.step()
```

Putting It All Together

Fashio MNIST



Putting It All Together

1. Hyperparameters
2. Handling data
3. Model, loss & optimizer
4. Training and inference
5. Overall training process

Putting It All Together | Hyperparams

```
# configurations
LR = 1e-3           # learning rate
BATCH_SIZE = 64     # batch size
EPOCHS = 30         # num epochs
INPUT_SIZE = 28     # input size (28x28 grayscale)
NUM_WORKERS = 2     # dataloader workers

device = 'cuda' if torch.cuda.is_available() else 'cpu'
```

Putting It All Together | Dataset

```
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor(),
)

val_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor(),
)
```

Putting It All Together | Dataset

```
train_loader = DataLoader(
    training_data,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=NUM_WORKERS,
)

val_loader = DataLoader(
    val_data,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=NUM_WORKERS,
)
```

Putting It All Together | Model, loss, optimizer

```
# define model
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(INPUT_SIZE*INPUT_SIZE, 512),
    nn.ReLU(),
    nn.Linear(512, 512),
    nn.ReLU(),
    nn.Linear(512, 10),
)

# move to correct device
model.to(device)
```

```
# define optimizer
optimizer = torch.optim.SGD(
    model.parameters(), lr=LR)
```

```
# define loss
criterion = nn.CrossEntropyLoss()
```

Putting It All Together | Training and inference

Training loop

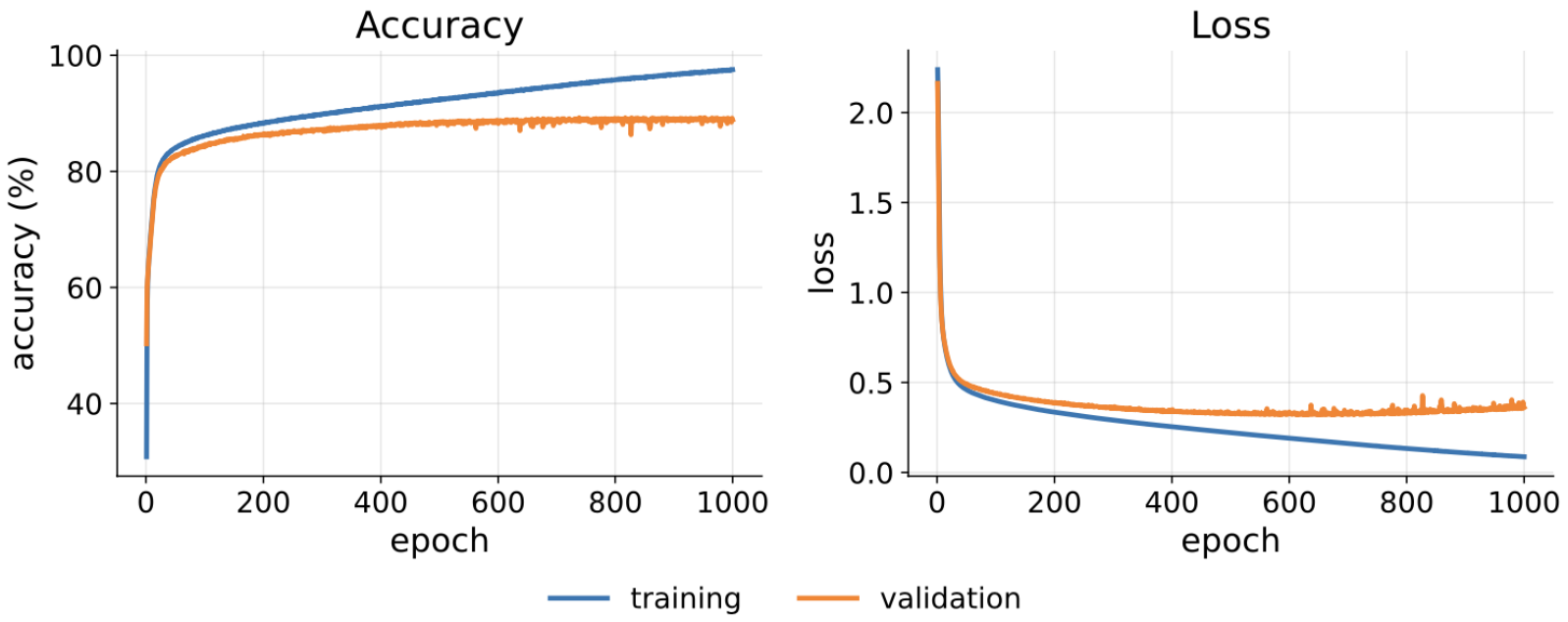
```
def train_loop(loader, model,
               criterion, optimizer):
    model.train()
    for X, y in loader:
        # move data to device
        X = X.to(device)
        y = y.to(device)
        # forward pass
        pred = model(X)
        loss = criterion(pred, y)
        # backward pass + update
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Inference loop

```
@torch.no_grad()
def inference_loop(loader, model,
                   criterion):
    model.eval()
    for X, y in loader:
        X = X.to(device)
        y = y.to(device)
        # forward only - no graph
        pred = model(X)
        loss = criterion(pred, y)
```

Putting It All Together

```
for t in range(EPOCHS):
    train_loop(train_loader, model, criterion, optimizer)
    inference_loop(val_loader, model, criterion)
```



Putting It All Together

```
for t in range(EPOCHS):
    train_loop(train_loader, model, criterion, optimizer)
    inference_loop(val_loader, model, criterion)
```

